ðŸ§ AI Brain System v6 â€” Full Project Analysis

What Is This?

This is a **brain-inspired AI reasoning system** built in Python. It does NOT train a model â€” instead it sits **on top** of existing LLMs (like NVIDIA-hosted GPT-OSS or OpenAI models) and adds a structured thinking pipeline with persistent memory, multi-perspective generation, scored evaluation, and single-pass refinement.

> *Think of it as a cognitive wrapper around an LLM: giving it memory, judgment, and improvement loops rather than just raw prompting.*

The Big Picture â€” What Happens When You Ask It a Question


Pipeline Architecture Diagram

Layer-by-Layer Breakdown

1. ðŸŽ›ï¸ Controller (`brain/controller.py`)

The **orchestrator**. It is the first thing called and coordinates everything else.

- Runs **memory cleanup** before reasoning (prunes old, low-quality memories)
- Decides the **strategy**: `direct` (fast) or `deliberate` (full pipeline)
  - Short queries (â‰¤ 4 tokens, no complex keywords) â†' `direct`
  - Longer or complex queries â†' `deliberate`
- Checks **case memory** for a reusable past solution
- Calls generator â†' evaluator â†' selector â†' refiner
- Persists the final answer back to memory

---

2. âš¡ Generator (`thinking/generator.py`)

Produces **3 candidate answers**, each from a different perspective:

Perspective	Focus
Scientific	Mechanisms, causality, evidence-based explanation
Logical	Structured assumptions, tradeoffs, consistency checks
Practical	Actionable steps, risks, implementation sequences

Each perspective uses **RAG (retrieved context)** from the knowledge base and memory to ground the answer. If an LLM API key is configured, it calls the model; otherwise it falls back to **heuristic templates**.

---

3. âš–ï¸ Evaluator (`thinking/evaluator.py`)

Scores each candidate answer on **3 criteria** (weighted):

Criterion	Weight

Correctness	40%
Clarity	30%
Usefulness	30%

- **Heuristic mode** (offline): uses token overlap, answer length, and perspective profiles
- **LLM Judge mode** (with API key): sends a structured prompt and gets a JSON rubric score back
- Threshold: score â‰¥ **8/10** to "pass"

---

4. ðŸŽ¯ Selector (`thinking/selector.py`)

Simple: picks the option with the **highest total score** from the evaluator.

---

5. ðŸ"§ Refiner (`thinking/refiner.py`)

Only runs in `deliberate` **strategy**. Takes the best answer + evaluation feedback and does **one improvement pass**:

- Re-retrieves relevant context
- Calls LLM (or heuristic fallback) to produce a stronger version
- The improved answer is only used if its new score is **higher** than the original

---

6. ðŸ§ Memory System (`memory/`)

File	Role
`vector_store.py`	Unified store combining knowledge base + episodic memory
`embedder.py`	Sparse local embeddings (TF-IDF style) OR remote OpenAI embeddings
`chunker.py`	Splits long texts into overlapping chunks (default: 120 tokens, 30 overlap)
`retriever.py`	Top-K retrieval with relevance + quality weighting; updates usage counts
`case_memory.py`	Stores/retrieves full problemâ†'answer pairs with cosine similarity matching
`persistent_store.py`	JSONL file I/O: load, upsert, touch (update access time), delete
`cleanup.py`	Prunes memories older than 30 days (episodic) or 45 days (case) with low usage

Memory lifecycle:

1. Answer chunks are stored with a quality score (only if score â‰¥ 7)
2. Each retrieval updates `last_accessed` + `usage_count`
3. Cleanup deletes stale, low-usage records

---

7. ðŸ—„ï¸ Knowledge Base (`knowledge_base/`)

A set of static Markdown files that serve as **local RAG grounding** for all pipeline stages:

- `brain_system_mapping.md` â€" maps architecture to brain analogies
- `fine_grained_rubrics.md` â€" rubric scoring guidance
- `generator_playbook.md` â€" generation strategy rules
- `refinement_playbook.md` â€" refinement instructions

- `rag_evaluation_flow.md` â€“ RAG-grounded evaluation flow
- `structured_answer_format.md` â€“ expected answer format
- `llm_evaluation_metric.md` â€“ LLM judge metric definitions
- `multi_agent_utility.md` â€“ future multi-agent notes

---

8. ðŸŒ API + UI (`api/` , `ui/` , `index.html`)

A **FastAPI server** exposing:

Endpoint	Method	Purpose
`/`	GET	Serves the browser chat UI (HTML)
`/health`	GET	Health check â†' `{"status": "ok"}`
`/think`	POST	Runs the full pipeline; body: `{"problem": "..."}`

Run with: `uvicorn api.main:app --reload` then open `http://127.0.0.1:8000`

---

9. âš™ï¸ Configuration (`config.py`)

Everything is controlled via environment variables or defaults:

Setting	Default	Control
Strategy token limit	4 words	`QUICK_QUERY_TOKEN_LIMIT`
Memory chunk size	120 tokens	`MEMORY_CHUNK_SIZE`
Quality threshold	7/10	`MEMORY_QUALITY_THRESHOLD`
Case reuse threshold	0.45 similarity	`CASE_REUSE_SIMILARITY_THRESHOLD`
Enable LLM generator	Off	`ENABLE_LLM_GENERATOR=1`
Enable LLM judge	Off	`ENABLE_LLM_JUDGE=1`
Enable LLM refiner	Off	`ENABLE_LLM_REFINER=1`
Enable remote embeddings	Off	`ENABLE_REMOTE_EMBEDDINGS=1`
Memory retention	30 days	`MEMORY_RETENTION_DAYS`

Two Modes of Operation

Mode	Condition	Behavior
Offline / heuristic	No API keys set	Uses template-based generation, heuristic scoring
Online / LLM-powered	API keys + env flags set	Uses real LLM calls for generation, judging, and refining

---

How To Run

```
# CLI
python main.py "Impact of AI on jobs"

# Browser UI
uvicorn api.main:app --reload
# â†’ open http://127.0.0.1:8000
```

Summary

This project is essentially a **self-improving Q&A reasoning engine** that:

1. Takes a problem as input
2. Generates 3 structured answers from different lenses
3. Scores and ranks them using a rubric judge
4. Refines the best one (for complex queries)
5. Persists the final answer to persistent memory so future similar questions can skip the full pipeline
6. Learns over time which answers were high quality and which can be pruned